



MATHÉMATIQUES ET INFORMATIQUE

Sciences

Université Paris Cité

2026

Rapport de projet



L2F2

DAVID JANISZEK

UNIVERSITE PARIS CITE



Rapport de projet ***L2F2***

Référence du document :	RP-LDF2	Validé par :	Gael MAHE
Version du document :	1.0	Validé le :	19/05/2026
Date du document :	19/05/26	Soumis le :	19/5/26
Auteur(s) :	KANGNI Dina BENALI Abdallah ACHAK Salim	Type de diffusion :	Document électronique (.pdf)
		Confidentialité :	Réservé aux étudiants UFR Maths-Info de l'université Paris Descartes

Les éléments d'authentification

Maître d'ouvrage :	Gael MAHE	Chef de projet :	Dina KANGNI
Date / Signature :		Date / Signature :	

Mots clés : *déquantification, arbre binaire, Julia, histogramme, sous-quantification*

Sommaire

1. Préambule	5
2. Contexte	5
3. Algorithme de déquantification.....	5
4. Architecture du code	6
5. Choix de l'équipe et difficultés rencontrées	8
5.1. Choix du langage	8
5.2. Documentation	8
5.3. Passage à l'échelle : gestion de la complexité	9
5.4. Visualisation de l'arbre.....	11
5.5. Déploiement	13
5.6. Gestion du projet.....	14
6. Ressources et outils utilisés	16
6.1. Communauté et documentation.....	16
6.2. Intelligence artificielle.....	16
7. Bilan et perspectives	16
8. GLOSSAIRE	18
9. REFERENCE	19
10. INDEX.....	19

1. Préambule

Ce rapport réalisé par l'équipe L2F2 en charge du développement d'une application de déquantification a pour objectif de présenter les choix finaux effectués par l'équipe au cours de la conception du projet.

L'équipe est composée de ACHAK Salim, BENALI Abdallah et KANGNI Dina. La réalisation de ce projet est faite sous la direction de l'encadrant Gaël Mahé, au sein de l'UFR Mathématiques-Informatique de l'Université Paris Cité, dans le cadre de la deuxième de licence d'informatique.

2. Contexte

La quantification consiste à représenter des valeurs numériques sur un nombre limité de bits. La sous-quantification à 1 bit supprime le bit de poids faible de chaque valeur. Par exemple, si nous voulons exprimer cela sur une base décimale ça reviendrait à dire que 3 devient 2, 5 devient 4, 7 devient 6.

Cette opération est irréversible en théorie car, une fois sous-quantifiée, la série originale ne peut pas être retrouvée directement. L'objectif du projet est d'étudier la possibilité de déquantifier c'est-à-dire retrouver la série originale sous certaines conditions. Ces conditions reposent sur l'exploitation de l'histogramme P des couples de valeurs successives de x . $P(i,j)$ comptabilise le nombre de fois que la paire $(x(n-1), x(n))$ prend les valeurs i et j . La connaissance de cet histogramme rend possible la reconstruction de séries candidates correspondant à x . Ce projet s'inscrit dans un contexte de recherche sur la quantification notamment dans les domaines du multimédia et des réseaux de neurones. Les travaux de Halalchi, Mahé et Jaïdane (ICASSP 2009)^[2] ont montré que sous certaines conditions il est possible de retrouver l'histogramme original à partir de l'histogramme sous-quantifié.

Les documents produits dans le cadre de ce projet sont un cahier des charges, un cahier de recette, un cahier de conception et un plan de test.

3. Algorithme de déquantification

Pour chaque valeur sous-quantifiée $xQ[n]$ la valeur originale $x[n]$ est soit $xQ[n]$ soit $xQ[n]+1$. L'ensemble de toutes les reconstructions possibles est représentable sous la forme d'un arbre binaire où chaque niveau correspond à une position dans la série et chaque nœud a au plus deux fils. P sert à contraindre cet arbre. À chaque étape, quand on envisage d'ajouter un nœud de valeur v comme fils d'un nœud de valeur u on vérifie si le couple (u,v) est encore disponible dans P c'est-à-dire si son compteur est supérieur

à 0. S'il est valide, on ajoute le nœud et on décrémente $P[(u,v)]$. Sinon la branche est élaguée.

L'algorithme parcourt l'arbre en profondeur. Chaque nœud hérite d'une copie de P de son père et décrémente le couple correspondant. Une fois ses fils créés le père libère son histogramme pour économiser la mémoire. Si aucun fils n'est compatible le nœud est élagué récursivement jusqu'au premier ancêtre ayant encore un fils vivant. Quand un nœud atteint le dernier niveau de xQ , la branche est extraite et sauvegardée comme solution.

4. Architecture du code

Le projet est organisé en trois modules principaux.

Le module *arbre.jl* définit les structures *Noeud* et *Arbre* ainsi que leurs fonctions. *construction.jl* est le programme auxiliaire : il lit x , génère xQ par masquage du bit de poids faible, construit P et sauvegarde les données.

dequantification.jl est le programme principal : il reçoit xQ et P , construit et élague l'arbre et génère les fichiers solutions.

L'interface graphique est implémentée dans *interface.jl* qui appelle les modules précédents via des fonctions de callback.

Cette architecture en modules indépendants a été choisie pour faciliter les tests unitaires. Chaque module peut être testé séparément. Le point faible est que *dequantification.jl* dépend de *arbre.jl* et que l'interface dépend des deux : un changement dans *arbre.jl* peut entraîner des modifications en cascade.

Par rapport au cahier de conception deux changements ont été faits.

On a d'abord rajouté le champ booléen *vivant*. Ce champ permet de marquer un nœud comme élagué sans le supprimer dans l'interface graphique. Cela évite les problèmes de modification de structure pendant le parcours récursif.

La fonction *elaguer!* a été modifiée en conséquence : au lieu de retirer physiquement un nœud de la liste d'enfants de son parent, elle met simplement le champ *vivant* à *false* et remonte récursivement si tous les enfants du parent sont morts. La fonction *compter_branche définie* dans *arbre.jl* ne compte que les feuilles avec *vivant* à *true*.

On a choisi de créer des scripts d'installation et de lancement propres à chaque système d'exploitation (Linux et Windows) afin de séparer clairement les environnements et de guider l'utilisateur pas à pas.

Le projet est distribué sous licence Apache 2.0 (licence disponible à la racine du projet). Il est donc autorisé de réutiliser notre code, de le modifier et de le redistribuer en mentionnant les noms des développeurs.

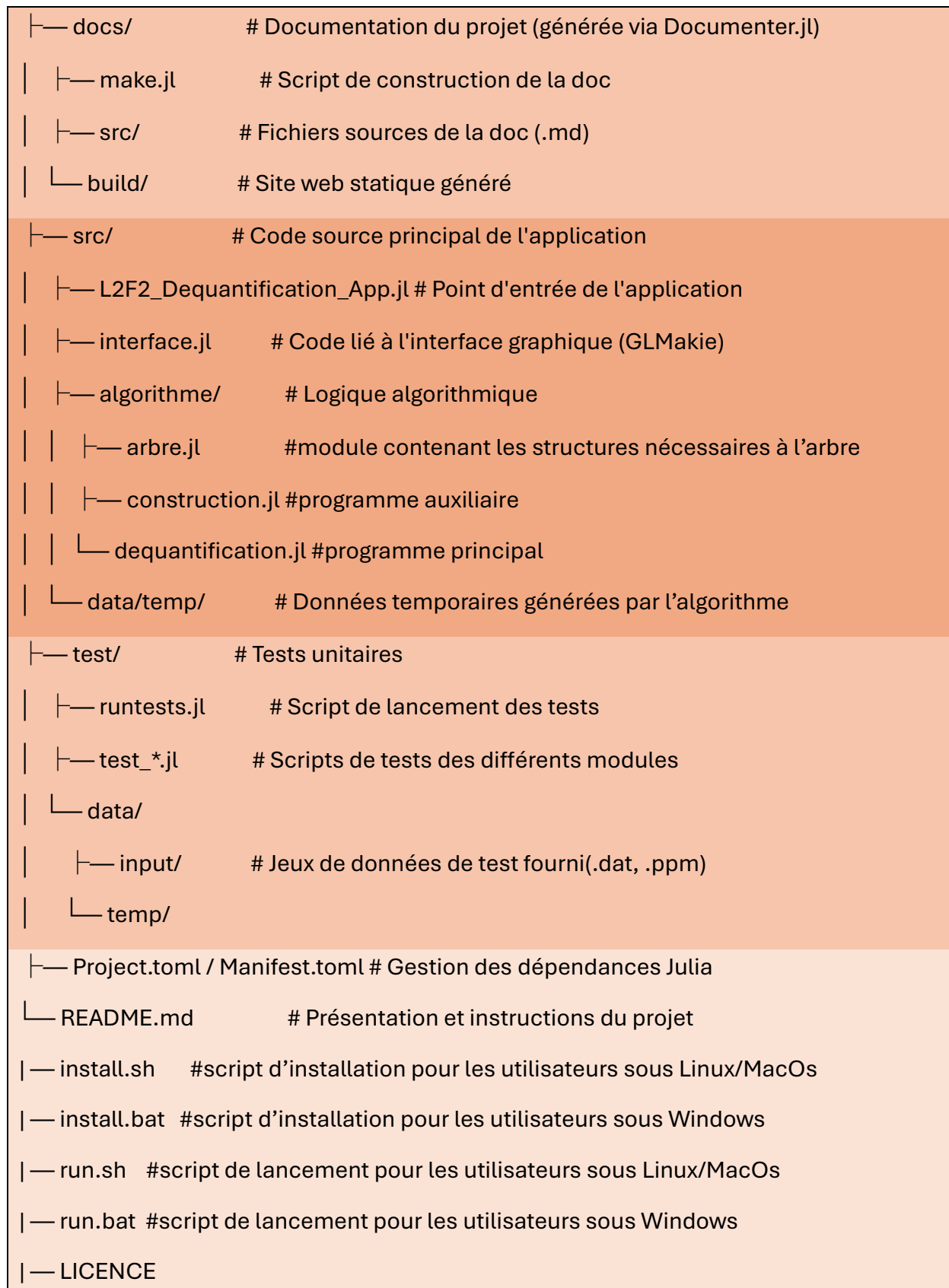


Figure 1 : représentation schématique et essentielle de l'architecture du projet

5. Choix de l'équipe et difficultés rencontrées

Cette section détaille les principaux choix techniques faits par l'équipe, les conséquences de ces choix et les difficultés rencontrées. Certains de ces choix ont nécessité de s'écarter du cahier de conception initial.

5.1. Choix du langage

Deux options s'offraient à nous pour le langage dans lequel allait être implémentée l'application : le C ou Julia.

Nous avons choisi Julia, en partie motivés par l'apprentissage d'un nouveau langage et de son application réelle.

Julia offre aussi une bibliothèque graphique intégrée (GLMakie) qui permet de d'implémenter l'interface et la visualisation de l'arbre assez simplement.

En C, il aurait fallu gérer la mémoire manuellement et intégrer des bibliothèques graphiques tierces ce qui aurait rendu le développement beaucoup plus lourd. Cependant, le principal problème est que Julia est plus lent que C sur des calculs intensifs.

Le déploiement a été difficile, particulièrement sur Windows.

En effet, on a été confronté à des fichiers DLL corrompus et des conflits avec les antivirus.

La compilation des bibliothèques graphiques prend également beaucoup de temps au premier lancement.

Un autre problème important a été la gestion des versions. On a commencé avec la dernière version actuelle de Julia à savoir la version 1.12. Elle s'avère cependant instable et cause des incompatibilités avec GLMakie. On a dû basculer vers Julia 1.10.11 une version LTS plus stable, ce qui a nécessité de vérifier à nouveau toutes les dépendances.

De plus, la communauté *Julia* étant plus petite que celle de C ou Python, trouver des réponses aux problèmes spécifiques à la version nous a demandé plus d'efforts.

5.2. Documentation

Dans un premier temps, la documentation du projet (cahier des charges, cahier de recette, cahier de conception, plan de test) était regroupée dans un dossier *doc* à la racine de notre architecture.

On a décidé de remplacer cette approche par un site généré avec *Documenter.jl*^[1], un package qui construit un site de documentation à partir des docstrings. On y a intégré toute la documentation du projet. Ce choix, nous a permis d'ajouter ou de modifier les *doctstrings* facilement. La documentation du site se met automatiquement à jour et est facilement consultable par les développeurs.

Le site est accessible localement. L'utilisateur peut y accéder à partir du dossier `./docs/build` en cliquant sur le fichier `index.html`.

5.3. Passage à l'échelle : gestion de la complexité

L'histogramme *P* représente la fréquence d'apparition des couples $(x[n-1], x[n])$ dans la série *x*.

La première approche était alors de le représenter sous la forme d'une matrice.

Par exemple, sur un fichier de 19 éléments, la matrice obtenue ressemble à ceci :

$x[n]$ $x[n-1]$	-2	-1	0	1	2
-2	0	1	1	1	0
-1	1	1	1	1	0
0	1	1	1	1	1
1	1	0	1	1	1
2	0	1	1	0	0

Figure 2 : matrice *P* extraite d'un fichier contenant une série à 19 éléments.

On constate que 28% des couples n'apparaissent jamais dans la série. Ce taux augmente avec la taille des séries : sur un fichier de 99994 éléments, c'est 95.07% des couples possibles qui n'apparaissent jamais. De plus, chaque nœud possède sa propre copie de l'histogramme ce qui rendrait cette représentation trop lourde en mémoire. On a donc fini par décider d'utiliser un dictionnaire qui stocke uniquement les couples existant dans la série.

On a été confronté à un autre défi. L'algorithme fonctionne correctement sur les petites séries d'une dizaine de valeurs c'est-à-dire que l'arbre se construit, s'élague et génère des solutions.

Cependant sur des séries plus grandes, l'arbre ne semble jamais finir de se développer. Visuellement, seul le côté gauche de l'arbre se construit et aucune solution n'est générée.

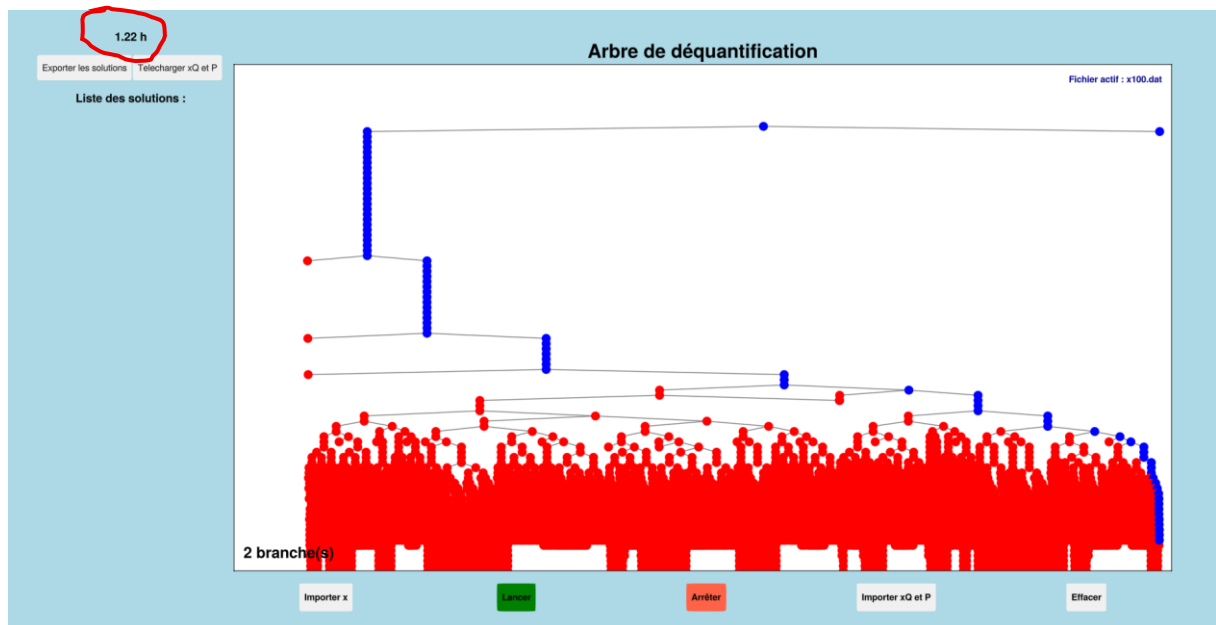


Figure 3 : Etat de l'interface graphique de l'application après près d'une heure trente d'exécution sur un fichier de 100 éléments.

En discutant avec l'encadrant, nous avons compris que le problème ne venait pas de notre implémentation mais de la manière dont nous avons imaginé la logique de notre fonction d'élagage et de la nature des données.

Lorsqu'un nœud ne peut plus générer de fils qui formeraient un couple compatible avec P, on le marque en rouge afin de signifier qu'il est élagué et on remonte dans l'arbre récursivement. L'algorithme parcourt donc l'arbre en profondeur en commençant par la main gauche.

Les séries de tests transmises par notre encadrant sont des jeux de données dans lesquelles les couples $(x(n-1), x(n))$ de faibles valeurs reviennent souvent dans l'histogramme P alors que les couples de grandes valeurs sont beaucoup plus rares. Ainsi, les combinaisons de couple autour de 0 restent valides pendant très longtemps car leur valeur dans P met du temps à s'épuiser.

L'algorithme descend donc très profondément du côté gauche avant de rencontrer une impasse. Lorsqu'il la rencontre il se met à remonter : il teste le frère droit, remonte encore si nécessaire, et ainsi de suite. Mais comme il est déjà descendu à une profondeur importante, l'espace à explorer pour remonter et basculer vers la main droite est de l'ordre de 2^n nœuds, avec n la taille de la série. Ce qui explique que notre algorithme semblait ne pas fonctionner.

Si on prend par exemple un fichier de 100 éléments générés aléatoirement, les couples de faibles et de grandes valeurs apparaissent de manière à peu près équivalente dans P. Aucune zone de l'histogramme ne concentre la plupart des occurrences. Ainsi, les branches rencontrent des impasses bien plus tôt. L'algorithme trouve une solution en quelques secondes.

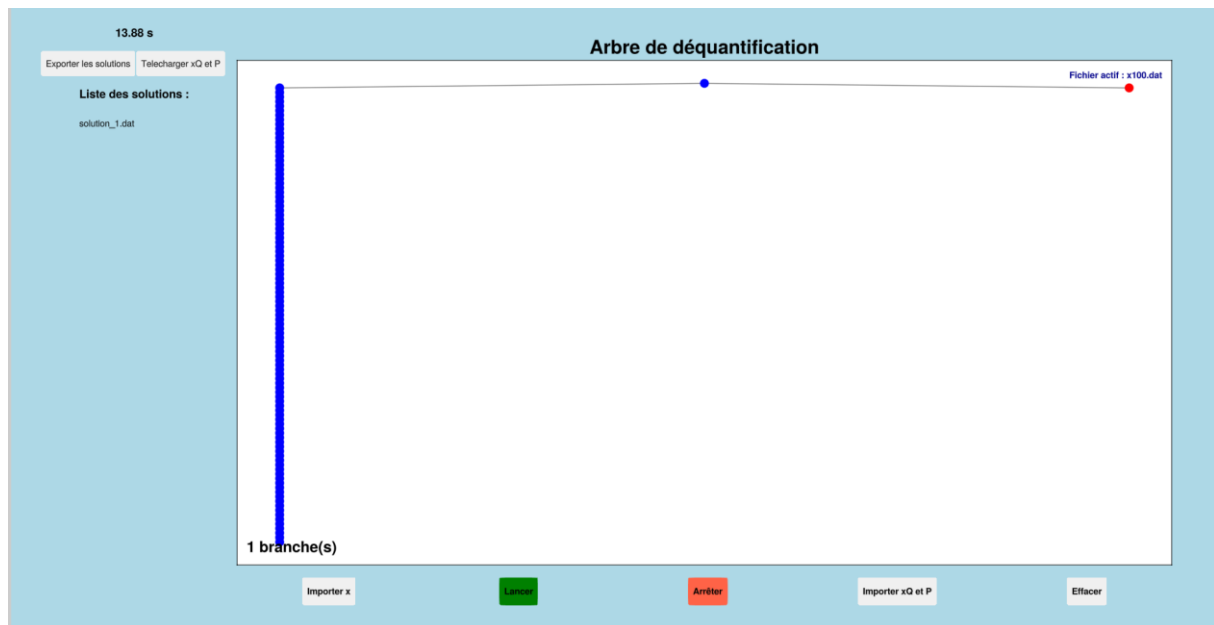


Figure 4 : État de l'interface graphique après environ 14 secondes d'exécution sur un fichier de 100 éléments générés aléatoirement

Nous pouvons comparer la *figure 2* et la *figure 3*. Alors qu'un fichier contenant 100 éléments dont l'histogramme vérifie les conditions de Widrow (*figure 2*) ne donne aucune solution après 1h30 d'exécution, un fichier de même taille généré aléatoirement de manière uniforme se termine en 14 secondes avec une solution trouvée.

On aurait pu modifier la logique du parcours non plus en explorant l'arbre dans l'ordre chronologique de la série, mais en commençant par les transitions les plus rares dans P, c'est-à-dire celles dont le compteur est le plus faible. Cela aurait permis de rencontrer des impasses bien plus tôt et donc d'élaguer l'arbre dès les premiers niveaux. On n'a pas pu exploiter cette logique faute de temps.

5.4. Visualisation de l'arbre

On voulait au départ implémenter le graphe qui allait nous permettre de visualiser l'arbre avec *SimpleDiGraph*, une structure de graphe fourni par la bibliothèque *GraphMakie* qui permet de dessiner un graphe sans se préoccuper du positionnement de chaque noeud. On a utilisé le layout *Buchheim* qui permet spécifiquement de construire un arbre en respectant visuellement la hiérarchie parent-enfant.

Cependant, sur *Julia 1.10*, ce layout se révélait être instable.

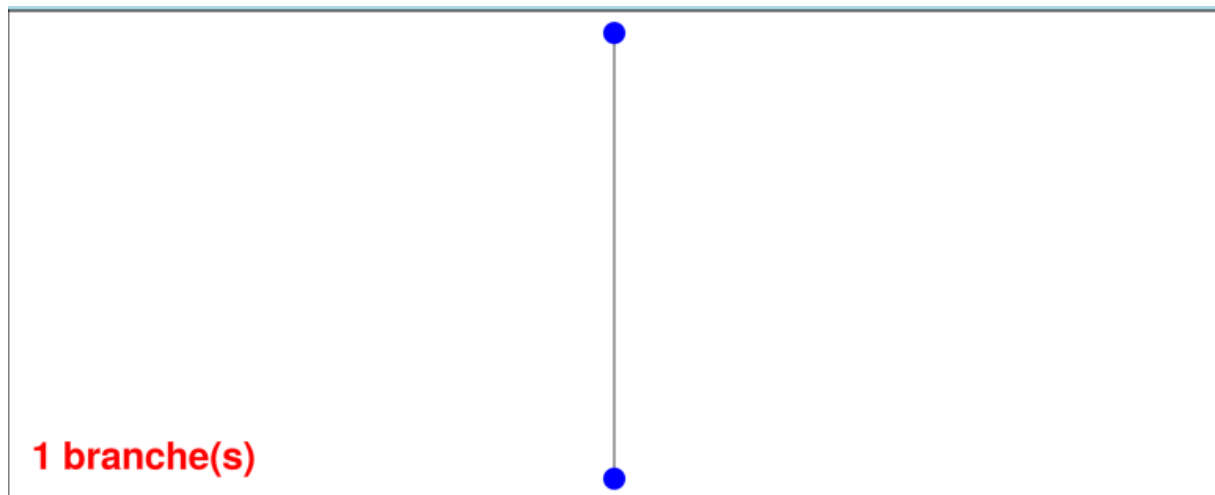


Figure 5 : Rendu de l'arbre avec le layout Buchheim sur le fichier x100.dat

On observe dans la figure 4 que le layout Buchheim produit un rendu incohérent.

Nous avons finalement décidé de construire l'arbre manuellement à l'aide de trois fonctions :

- *calculer_x* ! qui détermine la position horizontale des nœuds,
- *calculer_profondeur* ! qui détermine la position verticale des nœuds et
- *convertir_arbre* qui fait le lien entre l'arbre et le graphe *SimpleDiGraph*.

De plus, il était initialement prévu de voir visuellement les branches disparaître et se construire au fur et à mesure de l'élagage. Le problème est que cela générerait des incohérences entre la structure interne de l'arbre et sa représentation visuelle.

En effet, *SimpleDiGraph* identifie chaque nœud par un indice attribué dans l'ordre d'insertion. Lors de la suppression d'un nœud, ces indices sont réattribués, ce qui brise la correspondance entre les nœuds de notre structure et leurs positions dans le graphe. Nous avons donc décidé de ne pas montrer visuellement les suppressions mais plutôt de marquer en rouge, via le champ *vivant* ajouté à notre structure *Nœud*, les branches élaguées, quitte à perdre en lisibilité.

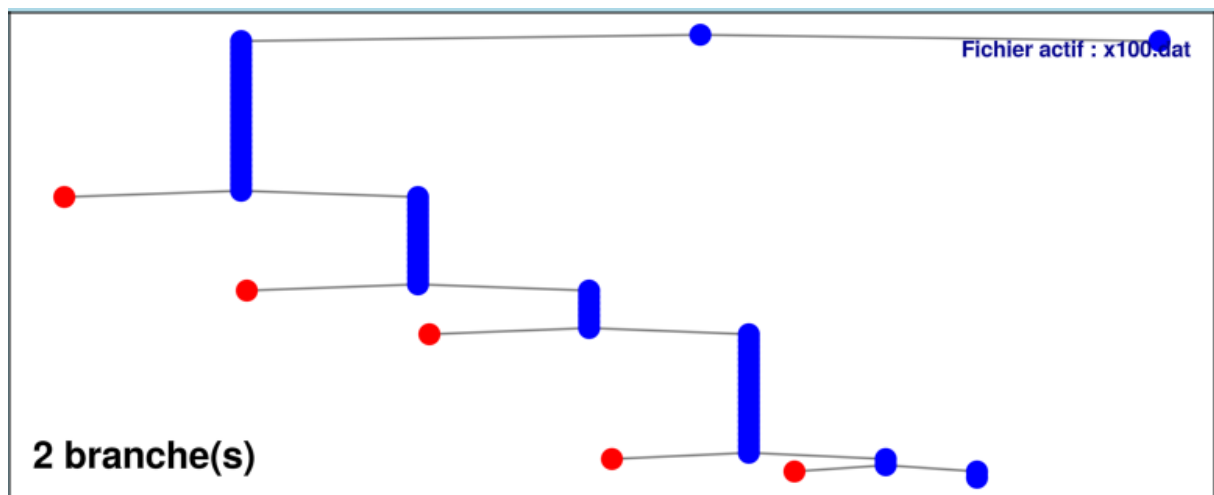


Figure 6 : Rendu de l'arbre sans le layout Buchheim et avec le marquage au lieu de l'élagage sur le fichier x100.dat

On observe dans la figure 5 que la représentation graphique de l'arbre est visuellement sous la forme d'une hiérarchie parent-enfant et que les nœuds élagués sont marqués en rouge.

Enfin, il était important pour nous d'afficher le pourcentage de branches restantes. On voulait le marquer en vert s'il était inférieur à 5% du nombre total de branches de l'arbre et en rouge sinon.

En effet, tout l'enjeu de notre projet est de savoir si l'approche que nous avons utilisée pour déquantifier, c'est-à-dire notre utilisation d'un histogramme P et de l'élagage d'un arbre binaire est efficace. Ce pourcentage aurait permis de mesurer cette efficacité.

C'est aussi pour mesurer cette efficacité que nous avons décidé de rajouter un chronomètre qui calcule le temps d'exécution de l'animation. Notre but étant d'évaluer l'algorithme aussi bien sur le plan algorithmique que temporel.

Cependant, calculer ce pourcentage nécessite de connaître le nombre total de branches initial, qui croît exponentiellement avec la taille de la série et devient rapidement inexact avec l'élagage.

Nous avons donc finalement choisi d'adopter une approche plus simple et de seulement afficher le nombre de branches restantes, mis à jour au fur et à mesure de l'animation.

5.5. Déploiement

Au départ, il était important pour l'équipe de produire un exécutable pour des raisons d'ergonomie. L'objectif était de permettre à l'utilisateur d'installer et d'utiliser l'application en effectuant le moins de manipulations techniques possible. Par exemple, on voulait éviter qu'il ait à installer *Julia* lui-même.

Il a donc été décidé d'utiliser *PackageCompiler.jl*, le package officiel de Julia pour générer un exécutable. Cependant, *PackageCompiler* produit un exécutable de plusieurs centaines de Mo car il contient, l'ensemble de l'environnement d'exécution de Julia (son moteur, ses artefacts, ses fichiers systèmes, ses DLL...) en plus des bibliothèques utilisées par notre programme. De plus, le temps de compilation est très lent surtout avec GLMakie, la bibliothèque graphique qu'on utilise.

L'ensemble de l'équipe a développé sous Windows. Nous avons rencontré des conflits de DLL (nos antivirus bloquaient ces fichiers, ce qui nous empêchait de travailler), des interruptions de communication entre processus internes, et surtout des crashes constants lors de l'initialisation de GLMakie.

Créer un exécutable n'apporte pas de confort supplémentaire puisqu'il est instable et disproportionné pour une application de 16,5 Mo.

Cette idée d'exécutable a donc été abandonnée au profit de scripts d'installation des dépendances (install.bat pour Windows et install.sh pour linux) et de lancement (run.sh pour linux et run.bat pour Windows). Ces scripts permettent à l'utilisateur d'installer les dépendances (ce qui peut prendre du temps) et par la suite de lancer l'application avec les scripts correspondants.

5.6. Gestion du projet

Dès le lancement du projet le 26 janvier 2026, l'équipe a défini ses moyens de communication. Discord a été principalement utilisé pour l'échange de documents et WhatsApp pour les réflexions autour du projet. SVN a été utilisé comme gestionnaire de version et la forge de l'université comme dépôt des documents rendus et de la communication avec l'encadrant Gaël Mahé. Toutes les semaines, une réunion avec ce dernier se tenait, et le chef de projet qui changeait devait rédiger un compte rendu.

Le tableau suivant détaille semaine par semaine la contribution de chaque membre de l'équipe au projet.

Semaine	Dina KANGNI	Abdallah BENALI	Salim Achak
S1 : 26/01 Définition	Prise de contact Définition des objectifs, travail de recherche	Prise de contact Définition des objectifs, travail de recherche	Prise de contact Définition des objectifs, travail de recherche
S2 : 02/02 Analyse	Définition des objectifs, travail de recherche Chef de projet	Définition des objectifs, travail de recherche	Définition des objectifs, travail de recherche

S3 : 09/02 Spécification <i>Rendu : cahier des charges</i>	Rédaction du cahier des charges	Rédaction du cahier des charges Chef de projet	Rédaction du cahier des charges
S4 : 16/02 Conception <i>Rendu : cahier de recette</i>	Rédaction du cahier de recette et correction du cahier des charges	Rédaction du cahier de conception et développement construction.jl	Rédaction du cahier de conception et développement arbre.jl+ fichiers de tests Chef de projet
S5 : 23/02 Développement <i>Rendu : conception détaillée</i>	Rédaction du cahier de conception, brouillons de développement et correction du cahier de recette Chef de projet	Rédaction du cahier de conception et développement construction.jl	Rédaction du cahier de conception, développement arbre.jl et rédaction des fichiers de test
S6- S9 : 09/03-06/03 Développement	Développement principal de interface.jl et dequantification.jl et modification de arbre.jl	Développement principal de construction.jl Chef de projet	Développement principal de arbre.jl, fichiers de tests et correction dequantifier.jl. Rédaction et mise à jour des fichiers de tests (test_arbre.jl, test_dequantification.jl) Corrections de dequantification.jl (résolution du bug de boucle infinie par remplacement de la suppression physique par le champ vivant::Bool)
S10 : 13/04 Intégration	Correction du code, ajout de fonctionnalités et test avec l'encadrant Correction du plan de test.	Correction du code, ajout de fonctionnalités et test avec l'encadrant	Correction du code et des fichiers de tests, ajout de fonctionnalités et tests avec l'encadrant, rédaction du plan de test Chef de projet
S11 : 20/04 Livraison	Dernières corrections du code. Chef de projet	Dernières corrections du code.	Dernières corrections du code.

6. Ressources et outils utilisés

6.1. Communauté et documentation

L'utilisation des ressources internet a été indispensable à la réalisation de ce projet. La documentation officielle de Julia nous a permis de nous familiariser avec la logique du langage.

Le site *StackOverflow* a été utile pour identifier les problèmes liés à *PackageCompiler*, pour la rédaction des scripts d'installation des dépendances et pour l'utilisation des packages.

La plateforme YouTube nous a beaucoup aidé dans l'implémentation de l'interface graphique et la prise en main de GLMakie.

Julia est un langage assez récent, ainsi sa communauté n'est pas aussi conséquente que celle d'autres langages comme Python ou C et des informations techniques ont donc été plus difficile à trouver, retardant parfois notre travail.

Cependant, le langage étant proche de Python, il a été simple de trouver des informations moins techniques.

6.2. Intelligence artificielle

Plusieurs IA ont été utilisées au cours du projet, principalement Claude et ChatGPT.

Ces outils ont servi d'une part, d'avoir une première vue d'ensemble sur le sujet du projet, c'est-à-dire sur la déquantification, et d'autre part à déboguer. Elles se révèlent être efficaces pour identifier rapidement les erreurs de logiques ou de syntaxe.

Elles nous ont par exemple beaucoup aidé à identifier les problèmes de compatibilité entre Julia et GLMakie.

7. Bilan et perspectives

Nous pensons que le projet a été mené avec succès malgré quelques difficultés d'implémentation du langage *Julia* et des pistes d'amélioration non implémentées.

En effet la compatibilité de Julia a été compliqué à gérer. Ses bibliothèques graphiques évoluent rapidement, et nous avons régulièrement rencontré des incompatibilités entre les versions (avec le layout *Buchheim* de *GraphMakie* par exemple).

De manière plus générale, c'est le développement sur Windows qui nous a posé beaucoup de problème. Les fichiers DLL de Julia se faisaient bloquer par les antivirus présents sur nos machines, ce qui empêchait par exemple le chargement de nos bibliothèques graphiques et a retardé notre codage. Nous avons été contraints de nous adapter en le désactivant pour certain ou en travaillant sous WSL pour d'autres, tout en faisant en sorte que le livrable reste fonctionnel sur Windows.

La génération d'un exécutable a également été abandonnée, en raison des fichiers DLL bloqués par les antivirus et du poids disproportionné du dossier produit par PackageCompiler pour une application de notre taille.

Mais il est possible que dans un futur proche on puisse utiliser ce package^[3]. Jeff Bezanson, co-créateur de Julia, et Gabriel Baraldi, ingénieur compilateur, ont présenté à la *JuliaCon 2024* *juliac*, un nouveau compilateur driver qui permettrait de ne conserver que le code réellement atteignable depuis le point d'entrée, réduisant la taille des binaires. Aujourd'hui, il s'agit encore d'une fonctionnalité expérimentale.

La deuxième difficulté majeure est celle du passage à l'échelle. Sur des séries longues, notre algorithme s'est révélé inefficace.

De plus, chaque nœud de l'arbre binaire possède sa propre copie indépendante de l'histogramme, décrétementée à chaque descente.

Une piste d'optimisation aurait été d'utiliser le concept d'Event Sourcing^[4]. Ce modèle consiste à ne stocker que les changements d'état plutôt que l'état complet à chaque instant. On aurait seulement sauvegardé dans chaque nœud le couple décrétementée à ce niveau. Pour connaître l'état de P en un nœud donné, il suffirait de remonter la branche depuis la racine et de rejouer les modifications.

Notre encadrant a également proposé une amélioration de l'élagage. Il s'agirait de parcourir l'arbre en commençant par les transitions les moins fréquentes dans P plutôt que dans l'ordre chronologique de la série, afin de rencontrer des impasses plus tôt et d'élaguer l'arbre dès les premiers niveaux

Ces pistes n'ont pas été implémentées faute de temps, mais elles constituent des améliorations envisageables.

Enfin, plusieurs fonctionnalités que nous aurions souhaité intégrer à l'interface n'ont pas pu être développées. Des boutons d'avance et de recul dans l'animation auraient permis à l'utilisateur de naviguer pas à pas dans la construction de l'arbre, mais leur implémentation nécessitait une gestion d'événements qu'on a pas pu développer. Nous avons également envisagé un bouton permettant d'afficher ou de masquer

l'histogramme directement dans l'interface, afin de donner à l'utilisateur une vision plus complète de l'état de l'algorithme à chaque étape.

8. GLOSSAIRE

Compilateur driver :

Programme qui orchestre toutes les étapes de la compilation.

Fichiers DLL :

Fichiers Windows contenant du code compilé et des données partageables entre plusieurs programmes.

Sous-quantification à 1 bit :

Opération qui supprime le bit de poids faible d'une valeur numérique, divisant par deux l'ensemble des valeurs possibles. Par exemple, 3 devient 2, 5 devient 4, 7 devient 6.

Déquantification :

Opération qui consiste à retrouver les données originales à partir de leur version quantifiée.

WSL :

Windows Subsystem for Linux, un environnement Linux qui s'exécute directement à l'intérieur de Windows.

Event Sourcing :

modèle de gestion d'état qui consiste à ne stocker que les modifications successives plutôt que l'état complet à chaque instant. Pour retrouver l'état courant, on rejoue les modifications depuis le début.

Elaguer :

supprimer une branche de l'arbre binaire lorsqu'elle ne peut plus mener à une solution valide, et remonter récursivement si le nœud parent devient à son tour sans enfant.

Conditions de Widrow :

L'ingénieur américain Bernard Widrow (1929–2025) énonce des conditions suffisantes pour retrouver l'histogramme original d'une série quantifiée. Soit x une série dont les valeurs suivent une loi de probabilité $p(x)$. On définit sa fonction caractéristique Φ comme la transformée de Fourier de $p(x)$.

Les conditions de Widrow sont les suivantes :

Si Φ est à support borné $[-\Omega, +\Omega]$ et que le pas de quantification λ vérifie $\Omega < \frac{1}{2\lambda}$, alors l'histogramme original peut être reconstruit à partir de l'histogramme du signal quantifié.

Dans le cas de la sous-quantification à k bits, cette condition devient $\Omega < \frac{1}{2^{k*2}}$, soit pour $k=1$: $\Omega < \frac{1}{4}$.

9. REFERENCE

- [1] Home · Documenter.jl. <https://documenter.juliadocs.org/stable/>
- [2] H. Halalchi, G. Mahé, M. Jaïdane, “Revisiting quantization theorem through audiowatermarking.”, *Proc. ICASSP 2009*, avril 2009, pp. 3361-364
- [3] Bezanson, J., & Baraldi, G. (2024, juillet 11). New ways to compile julia juliacon 2024. <https://pretalx.com/juliacon2024/talk/FUTLND/>
- [4] Flower, M. (2005, décembre, 12). Event sourcing. <https://martinfowler.com/eaDev/EventSourcing.html>

10. INDEX

<i>arbre.jl</i> , 6, 14	histogramme, 2, 5, 6, 8, 9, 12, 16
<i>dequantification.jl</i> , 6, 14	Intégration, 14
Développement, 14	<i>interface.jl</i> , 6, 14
DLL, 7, 13, 16	<i>Julia</i> , 2, 7, 10, 12, 13, 15, 16
documentation, 3, 7, 15, 16	package, 7, 13, 16
élagage, 9, 11, 12, 16	quantification, 2, 5
GLMakie, 7, 13, 15	<i>SimpleDiGraph</i> , 10, 11